

Managing Transaction Processing

In this chapter, you'll learn what transactions are and how to use `COMMIT` and `ROLLBACK` statements to manage transaction processing.

Understanding Transaction Processing

Note

Not All Engines Support Transactions—As explained in Chapter 21, “Creating and Manipulating Tables,” MySQL supports the use of several underlying database engines. Not all engines support explicit management of transaction processing, as will be explained in this chapter. The two most commonly used engines are MyISAM and InnoDB. The former does not support explicit transaction management and the latter does. This is why the sample tables used in this book were created to use InnoDB instead of the more commonly used MyISAM. If you need transaction processing functionality in your applications, be sure to use the correct engine type.

Transaction processing is used to maintain database integrity by ensuring that batches of MySQL operations execute completely or not at all.

As explained back in Chapter 15, “Joining Tables,” relational databases are designed so data is stored in multiple tables to facilitate easier data manipulation, management, and reuse. Without going into the hows and whys of relational database design, take it as a given that well-designed database schemas are relational to some degree.

The orders tables you've been using in prior chapters provide a good example of this. Orders are stored in two tables: `orders` stores orders, and `orderitems` stores the individual items ordered. These two tables are related to each other using unique IDs called *primary keys* (as discussed in Chapter 1, “Understanding SQL”). These tables, in turn, are related to other tables containing customer and product information.

The process of adding an order to the system is as follows:

1. Check if the customer is already in the database (that is, present in the `customers` table). If not, add them.
2. Retrieve the customer's ID.
3. Add a row to the `orders` table that associates it with the customer ID.
4. Retrieve the new order ID assigned in the `orders` table.
5. Add one row to the `orderitems` table for each item ordered and associate it with the `orders` table by the retrieved ID (and with the `products` table by the product ID).

Now imagine that some database failure (for example, lack of disk space, security restrictions, table locks) prevents this entire sequence from completing. What happens to your data?

Well, if the failure occurs after the customer is added and before the `orders` table is added, there is no real problem. It is perfectly valid to have customers without orders. When you run the sequence again, the inserted customer record will be retrieved and used. You can effectively pick up where you left off.

But what if the failure occurs after the `orders` row is added and before the `orderitems` rows are added? Now you have an empty order sitting in your database.

Worse, what if the system fails during the process of adding the `orderitems` rows? Now you end up with a partial order in your database, but you don't know it.

How do you solve this problem? This is where transaction processing comes in. *Transaction processing* is a mechanism used to manage sets of MySQL operations that must be executed in batches to ensure that databases never contain the results of partial operations. With transaction processing, you can ensure that sets of operations are not aborted mid-processing; they either execute in their entirety or not at all (unless explicitly instructed otherwise). If no error occurs, the entire set of statements is committed (written) to the database tables. If an error does occur, a rollback (undo) can occur to restore the database to a known and safe state.

So, in the context of this example, here is how the process works:

1. Check if the customer is already in the database. If not, add them.
2. Commit the customer information.
3. Retrieve the customer's ID.
4. Add a row to the `orders` table.
5. If a failure occurs while adding the row to `orders`, roll back.
6. Retrieve the new order ID assigned in the `orders` table.
7. Add one row to the `orderitems` table for each item ordered.
8. If a failure occurs while adding rows to `orderitems`, roll back all the `orderitems` rows added and the `orders` row.
9. Commit the order information.

When working with transactions and transaction processing, you need to know a few important terms:

- **Transaction:** A block of SQL statements
- **Rollback:** The process of undoing specified SQL statements
- **Commit:** The process of writing unsaved SQL statements to the database tables
- **Savepoint:** A temporary placeholder in a transaction set to which you can issue a rollback (as opposed to rolling back an entire transaction)

Controlling Transactions

Now that you know what transaction processing is, let's look at what is involved in managing transactions.

The key to managing transactions involves breaking your SQL statements into logical chunks and explicitly stating when data should be rolled back and when it should not.

The MySQL statement used to mark the start of a transaction is:

► Input

```
| START TRANSACTION;
```

Using ROLLBACK

The MySQL ROLLBACK command is used to roll back (undo) MySQL statements, as shown in this statement:

► Input

```
| SELECT * FROM ordertotals;  
| START TRANSACTION;  
| DELETE FROM ordertotals;  
| SELECT * FROM ordertotals;  
| ROLLBACK;  
| SELECT * FROM ordertotals;
```

► Analysis

Granted, this is not a particularly useful example, but it does help demonstrate the logical flow of a SQL transaction. This example starts by displaying the contents of the `ordertotals` table (which was populated in Chapter 24, “Using Cursors”). First, a `SELECT` is performed to show that the table is not empty. Then a transaction is started, and all of the rows in `ordertotals` are deleted with a `DELETE` statement. Another `SELECT` verifies that, indeed, `ordertotals` is empty. Then a `ROLLBACK` statement is used to roll back all statements until `START TRANSACTION`, and the final `SELECT` shows that the table is no longer empty.

Obviously, `ROLLBACK` can only be used within a transaction (after a `START TRANSACTION` command has been issued).

Tip

Which Statements Can You Roll Back? Transaction processing is used to manage INSERT, UPDATE, and DELETE statements. You cannot roll back SELECT statements. (There would not be much point in doing so anyway.) You cannot roll back CREATE or DROP operations. These statements may be used in a transaction block, but if you perform a rollback, they will not be undone.

Using COMMIT

MySQL statements are usually executed and written directly to the database tables. A commit (write or save) operation that happens automatically is known as an *implicit commit*.

Within a transaction block, however, commits do not occur implicitly. To force an explicit commit, the **COMMIT** statement is used, as shown here:

► Input

```
START TRANSACTION;  
DELETE FROM orderitems WHERE order_num = 20010;  
DELETE FROM orders WHERE order_num = 20010;  
COMMIT;
```

► Analysis

In this example, order number 20010 is deleted from the system entirely. Because this involves updating two database tables, **orders** and **orderitems**, a transaction block is used to ensure that the order is not partially deleted; you wouldn't want data deleted from one table but not the other. The final **COMMIT** statement writes the change only if no error occurred. If the first **DELETE** worked but the second failed, the **DELETE** would not be committed; rather, it would effectively be automatically undone.

Note

Implicit Transaction Closes After a **COMMIT** or **ROLLBACK** statement has been executed, the transaction is automatically closed (and future changes are implicitly committed).

Using Savepoints

Simple **ROLLBACK** and **COMMIT** statements enable you to write or undo an entire transaction. Although this works for simple transactions, more complex transactions might require partial commits or rollbacks.

For example, the process of adding an order described previously is a single transaction. If an error occurs, you only want to roll back to the point before the **orders** row was added. You do not want to roll back the addition to the **customers** table (if there was one).

To support the rollback of partial transactions, you must be able to put placeholders at strategic locations in the transaction block. Then, if a rollback is required, you can roll back to one of the placeholders.

These placeholders are called *savepoints*, and to create one, you use the `SAVEPOINT` statement, as follows:

► Input

```
| SAVEPOINT delete1;
```

Each savepoint has a unique name that identifies it so that, when you roll back, MySQL knows where you are rolling back to. To roll back to the savepoint just created, use the following:

► Input

```
| ROLLBACK TO delete1;
```

Tip

The More Savepoints, the Better You can have as many savepoints as you like within your MySQL code, and the more, the better. Why? Because the more savepoints you have, the more flexibility you have in managing rollbacks exactly as you need to.

Note

Releasing Savepoints A savepoint is automatically released after a transaction completes (that is, after a `ROLLBACK` or `COMMIT` is issued). As of MySQL 5, you can also explicitly release a savepoint by using `RELEASE SAVEPOINT`.

Changing the Default Commit Behavior

As already explained, the default MySQL behavior is to automatically commit any and all changes. In other words, any time you execute a MySQL statement, that statement is actually being performed against the tables, and the changes are made immediately. To instruct MySQL to not automatically commit changes, you need to use the following statement:

► Input

```
| SET autocommit=0;
```

► Analysis

The `autocommit` flag determines whether changes are committed automatically without requiring a manual `COMMIT` statement. Setting `autocommit` to 0 (false) instructs MySQL to not automatically commit changes (until the flag is set back to 1, or true).

Note

Flag Is Connection Specific The autocommit flag is per connection, not server-wide.

Summary

In this chapter, you learned that transactions are blocks of SQL statements that must be executed as a batch. You learned how to use the `COMMIT` and `ROLLBACK` statements to explicitly manage when data is written and when it is undone. You also learned how to use savepoints to provide a greater level of control over rollback operations.